

2026-2 컴퓨터 비전

기초 코드 실습 2

담당교수 최재영
담당조교 한상훈

기초 코드 실습 목표

- 1주차

- 기본 python 라이브러리를 통한 list 변환
- Numpy 라이브러리를 통한 list 변환, 이미지 변환
- Opencv 라이브러리를 통한 이미지 변환
- Image masking
- OpenCV 라이브러리

- 2주차

- OpenCV Feature Detection
- Scikit-learn Dataset Preprocessing

In Week 1: Python Data Structures

- List

```
myList = [1, 2, 3]
```

- Tuple

```
myTuple = (1, 2, 3)
```

- Dictionary

```
myDict = {'a':1, 'b':2, 'c':3}
```

- Python에서 여러 개의 데이터를 다루는 자료형들 - List, Tuple, Dictionary
 - 선언하는 방법에 따라 자료형이 달라지는 것을 볼 수 있음
 - List: [...], Tuple: (...), Dictionary: { 'key1':value1, 'key2':value2, ... }

In Week 1: Numpy

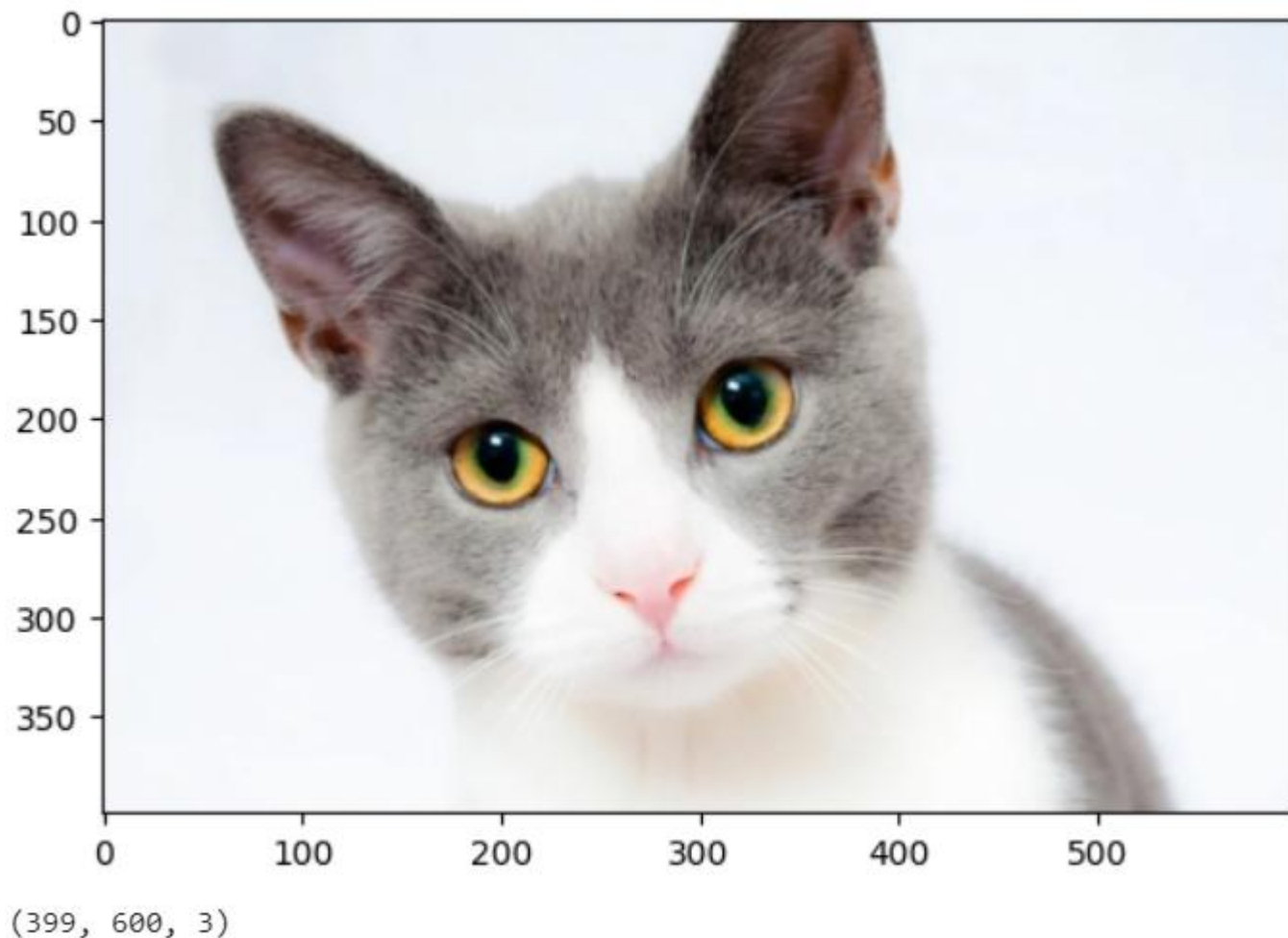
```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

path = 'Desktop/cat.jpg' # input your image path

image_pil = Image.open(path)
image = np.array(image_pil)

plt.imshow(image)
plt.show()
print(image.shape)
```

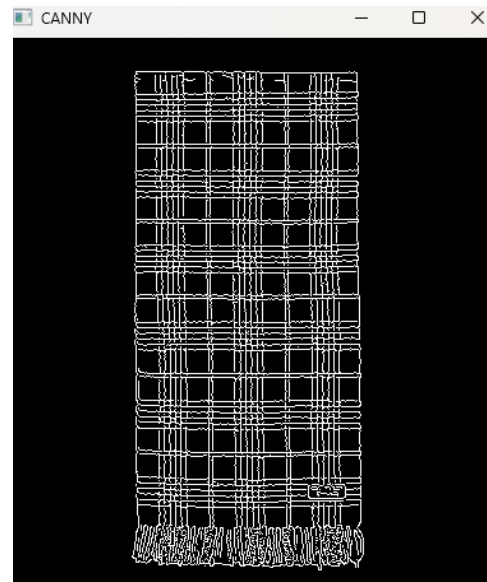
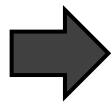
샘플 코드



In Week 1: Canny Edge Detection

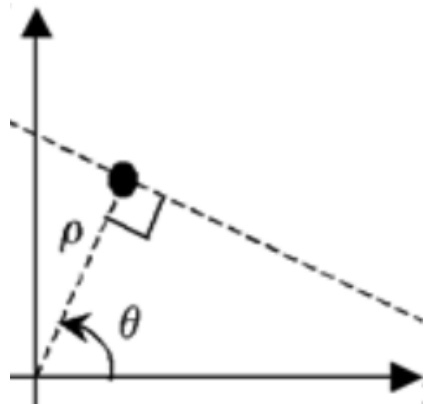
- Canny Edge Detection → multi-stage algorithm
 1. Gaussian 필터를 적용하여 노이즈 제거
 2. Pixel의 gradient 계산 (magnitude & orientation)
 3. Non-maximum Suppression을 적용하여 최대 강도의 gradient direction
 4. 실제 edge와 noise들을 Threshold 2개 (min, max)를 이용하여 hysteresis thresholding을 통해 정확한 edge 검출
- 검출된 edge 영역은 이후 line, circle 등 검출에 활용 가능

```
canny = cv2.Canny(img, 100, 200)
cv2.imshow("CANNY", canny)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Line Detection: OpenCV Hough Transform

- 이미지 내의 직선 찾기. 직선 = edge영역이 일정 이상 이어진 부분
 - 원점에 대해 rho (거리), theta(각도)로 직선을 표현하고, threshold 이상일 때 검출
- `cv2.HoughLines(img, rho, theta, threshold, lines, srn=0, stn=0, min_theta, max_theta)`
- rho, theta : 직선 검출의 세밀함 조절, 몇 rho/theta마다 검사할 것인지 지정
- threshold : 직선으로 판단하기 위한 최소 픽셀(edge 영역) 수



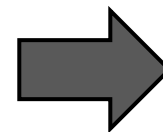
```
img = edge.copy()

#Gray변환 후 엣지 검출
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)

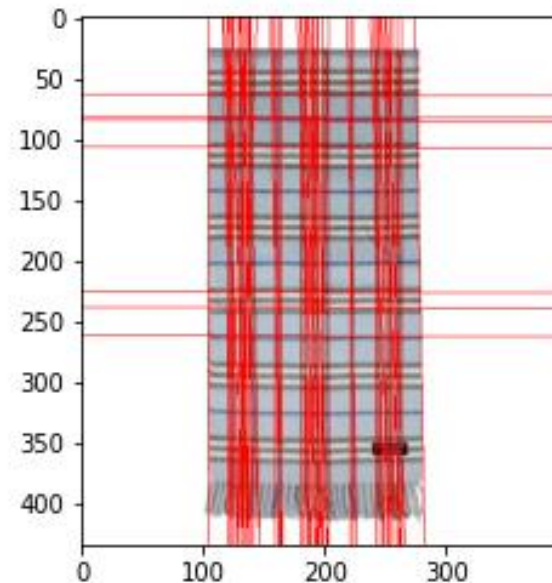
Edge 검출 이미지에서, 1픽셀 & 1도(=pi/180)마다 검사
#허프 라인 검출, 직선으로 판단할 최소한의 동일 개수 = 150px
lines = cv2.HoughLines(edges, 1, np.pi/180, 150)
print(lines.shape) #(N, 1, 2)

for line in lines:
    rho, theta = line[0] #Distance, angle return
    a = np.cos(theta)
    b = np.sin(theta)
    x0 = a*rho
    y0 = b*rho #x, y 절편 coord
    x1 = int(x0 + 1000*(-b))
    y1 = int(y0 + 1000*(a))
    x2 = int(x0 - 1000*(-b))
    y2 = int(y0 - 1000*(a))
    #line 기준점 연산
    cv2.line(img, (x1, y1), (x2, y2), (0, 0, 255), 1)

imgshow(img)
```



(66, 1, 2)



Line Detection: OpenCV Hough Transform

- Hough Transform: 모든 픽셀에 대해 수행 → 많은 시간 소모
- `cv2.HoughLinesP(img, rho, theta, threshold, lines, minLineLength, maxLineGap)`
→ 무작위로 선택한 edge 픽셀에 대해서만 직선 검출
→ edge 픽셀이 이어지는 구간을 선분으로 검출하고 반환 (`cv2.HoughLines`는 길이가 무한인 직선)

* 원을 찾고 싶다면 → `cv2.HoughCircles(img, method, dp, minDist, circles, param1, param2, minRadius, maxRadius)`

```
img = edge.copy()

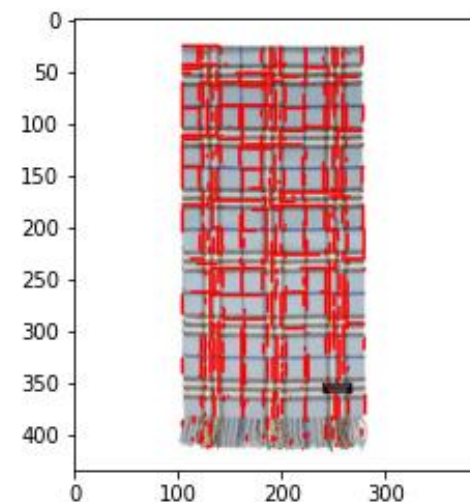
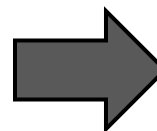
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
minLineLength = 100
maxLineGap = 0

lines = cv2.HoughLinesP(edges, 1, np.pi/360, 100, minLineLength, maxLineGap)
print(lines.shape) #(N, 1, 4 - x1, y1, x2, y2: 시작과 끝 좌표)

for line in lines:
    for x1, y1, x2, y2 in line:
        cv2.line(img, (x1, y1), (x2, y2), (0, 0, 255), 2)

imgshow(img)
```

528개의 line, 각각 x1, y1, x2, y2 좌표 4개
(528, 1, 4)



OpenCV Harris Corner Detection

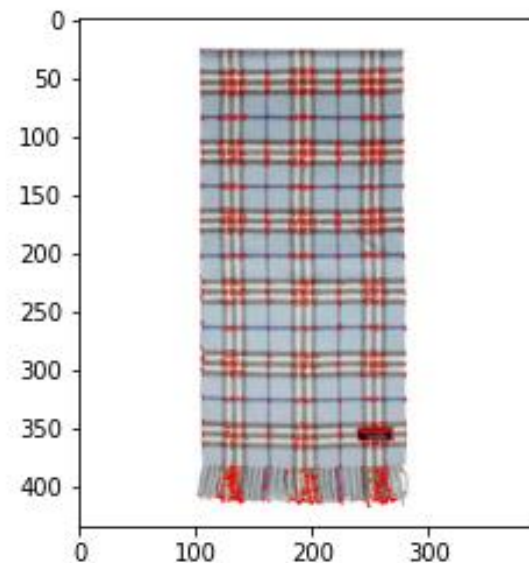
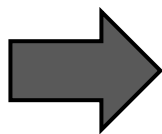
- Corner를 찾고 싶을때에는?
- `cv2.cornerHarris(img, blockSize (주변부 사이즈), kSize, freeParameter)`

```
img = edge.copy()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
dst = cv2.cornerHarris(gray, 2, 3, 0.04)

#Threshold for an optimal value
img[dst > 0.01 * dst.max()] = [0, 0, 255]

imgshow(img)
```



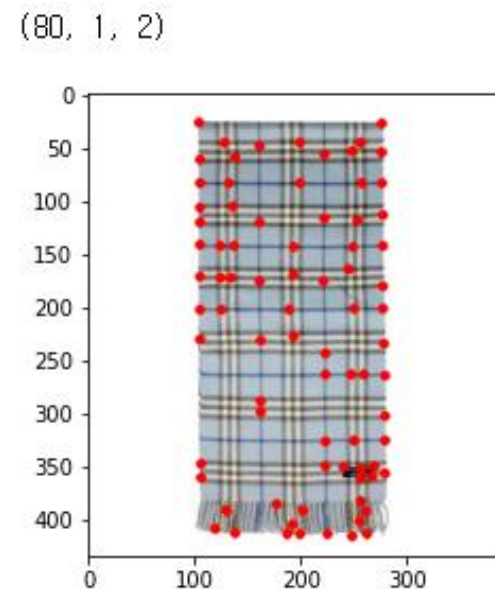
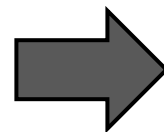
OpenCV Shi & Tomashi Detection

- Harris corner detection의 개선된 버전
- `cv2.goodFeaturesToTrack(img, maxCorners, qualityLevel, minDistance)`

```
img = edge.copy()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
dst = cv2.goodFeaturesToTrack(gray, 80, 0.01, 10)
print(dst.shape)
for corner in dst:
    corner = np.uint16(np.around(corner))
    cv2.circle(img, corner[0], 2, (0,0,255), 5)

imgshow(img)
```



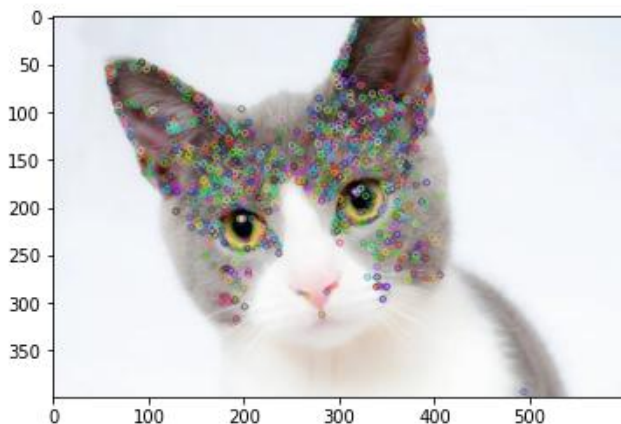
OpenCV Feature extraction

- OpenCV를 통해 다양한 feature detector들을 사용할 수 있음.
- Detector = cv2.??Detector_create()
- Keypoint = detector.detect(img, mask = None)
- Img = cv2.drawKeypoints(img, keypoint, outimg, color, flags)

```
img = cat.copy()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

detector = cv2.GFTTDetector_create()
keypoint = detector.detect(gray, None)
imgfe = cv2.drawKeypoints(img, keypoint, None)

imshow(imgfe)
```

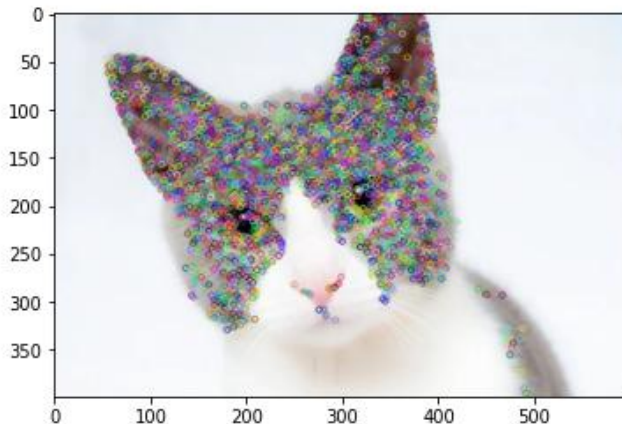


GFTTDetector (Shi-Tomashi 기반)

```
img = cat.copy()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

detector = cv2.FastFeatureDetector_create()
keypoint = detector.detect(gray, None)
imgfe = cv2.drawKeypoints(img, keypoint, None)

imshow(imgfe)
```

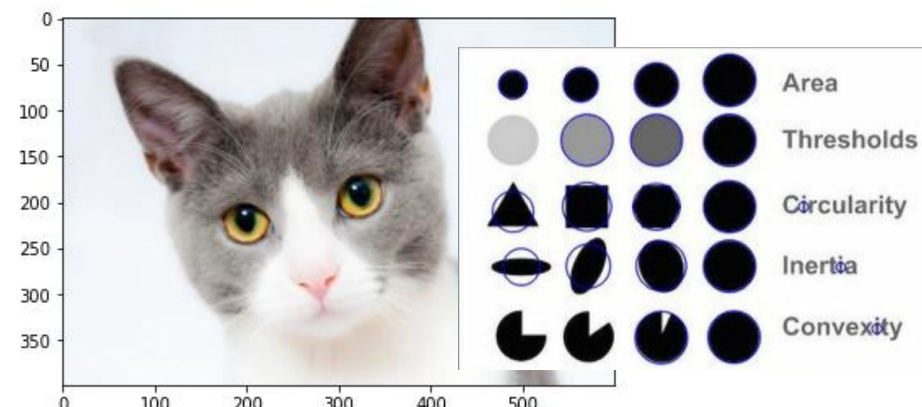


FastFeatureDetector (corner)

```
img = cat.copy()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

detector = cv2.SimpleBlobDetector_create()
#SimpleBlobDetector_Params로 별도 지정 가능!
keypoint = detector.detect(gray, None)
imgfe = cv2.drawKeypoints(img, keypoint, None)

imshow(imgfe)
```



SimpleBlobDetector (feature X)

OpenCV Image Pyramid

- Gaussian pyramid : 가우시안 필터로 블러 처리 후 해상도를 절반씩 줄여 피라미드 생성
- `cv2.pyrDown(img)` : 이미지 피라미드 축소
- `cv2.pyrUp(img)` : 이미지 피라미드 확대

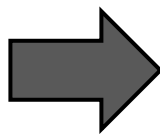
```
img = fox.copy()
down = cv2.pyrDown(img) # img x 1/4
up = cv2.pyrUp(img) # img x 4

imgarr = [img, down, up]

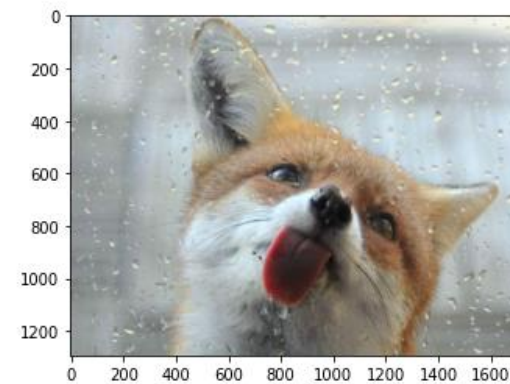
for i in imgarr:
    imgshow(i)
    print(i.shape)
```



(646, 880, 3)



(323, 440, 3)



(1292, 1760, 3)

OpenCV Image Pyramid

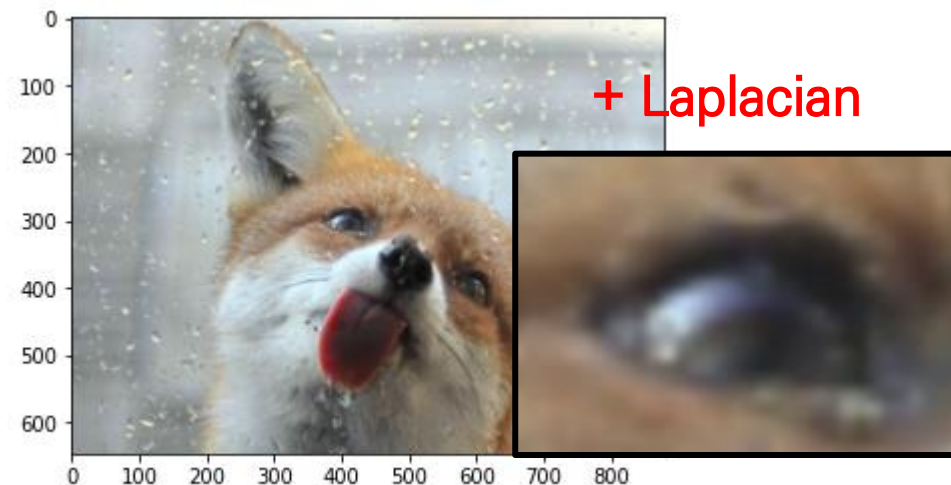
- **Laplacian pyramid** :
원본과 원본 → pyrDown → pyrUp 이미지의 차이(=손실된 디테일)를 저장한 피라미드
- Laplacian pyramid를 통해 이미지 디테일 복원

```
img = fox.copy()
down = cv2.pyrDown(img) # img x 1/4
up = cv2.pyrUp(down) # img x 4
imgshow(up)
```



```
img = fox.copy()
down = cv2.pyrDown(img)
up = cv2.pyrUp(down)

# 원본에서 확대 subtract
laplacian = cv2.subtract(img, up)
# 확대 이미지에 라플라시안 더해서 복원
img = up + laplacian
imgshow(img)
```





- Scikit-learn → Python의 머신러닝 라이브러리
- Scikit-image → 이미지 처리 알고리즘 패키지

Data preprocessing with Scikit-learn

- 예시 Dataset 불러오기 (Digits)

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
import numpy as np

digits = load_digits()

print(digits.images.shape)
print(digits.images[0].shape)
print(digits.data.shape)

fig, axes = plt.subplots(2, 5, figsize=(10, 5))

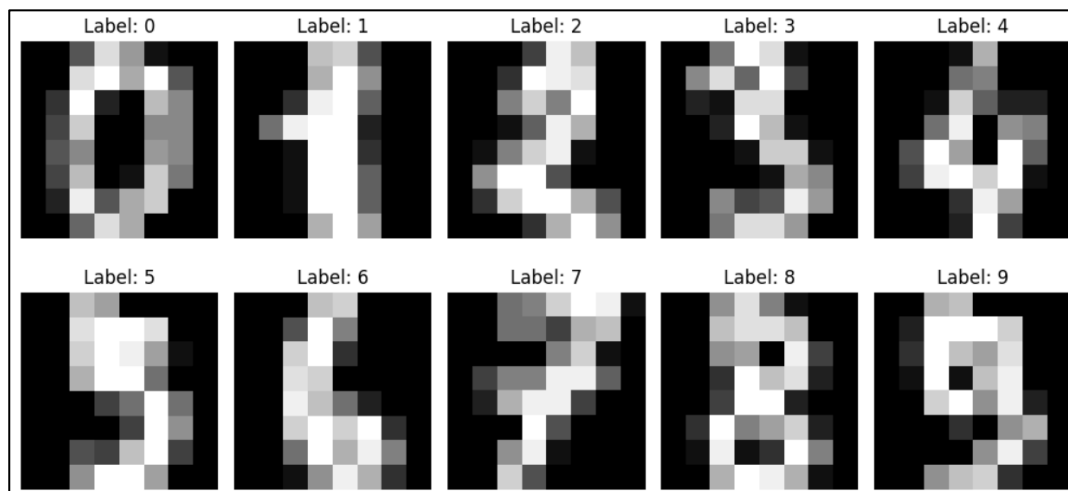
for digit in range(10):
    # 해당 숫자의 첫 번째 이미지 선택
    idx = np.where(digits.target == digit)[0][0]

    ax = axes.flat[digit]
    ax.imshow(digits.images[idx], cmap='gray')
    ax.set_title(f"Label: {digit}")
    ax.axis('off')

plt.tight_layout()
plt.show()
```

(1797, 8, 8)
(8, 8)
(1797, 64)

데이터 수: 1794
이미지 크기: (8, 8)
데이터 형식: (64,)



Data preprocessing with Scikit-learn

- Train set / Test set 나누기

sklearn.model_selection.train_test_split

```
sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)
```

[source]

```
from sklearn.model_selection import train_test_split
```

```
n_digits = len(digits.images)
```

Full Data

```
X = digits.data  
y = digits.target
```

```
print(X.shape)
```

```
print(y.shape)
```

```
print("-----")
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Train 80% / Test 20%

```
print(X_train.shape)
```

```
print(X_test.shape)
```

Train input (X_train)

```
(1797, 64)
```

Test input (X_test)

```
(1797,)
```

Train label (y_train)

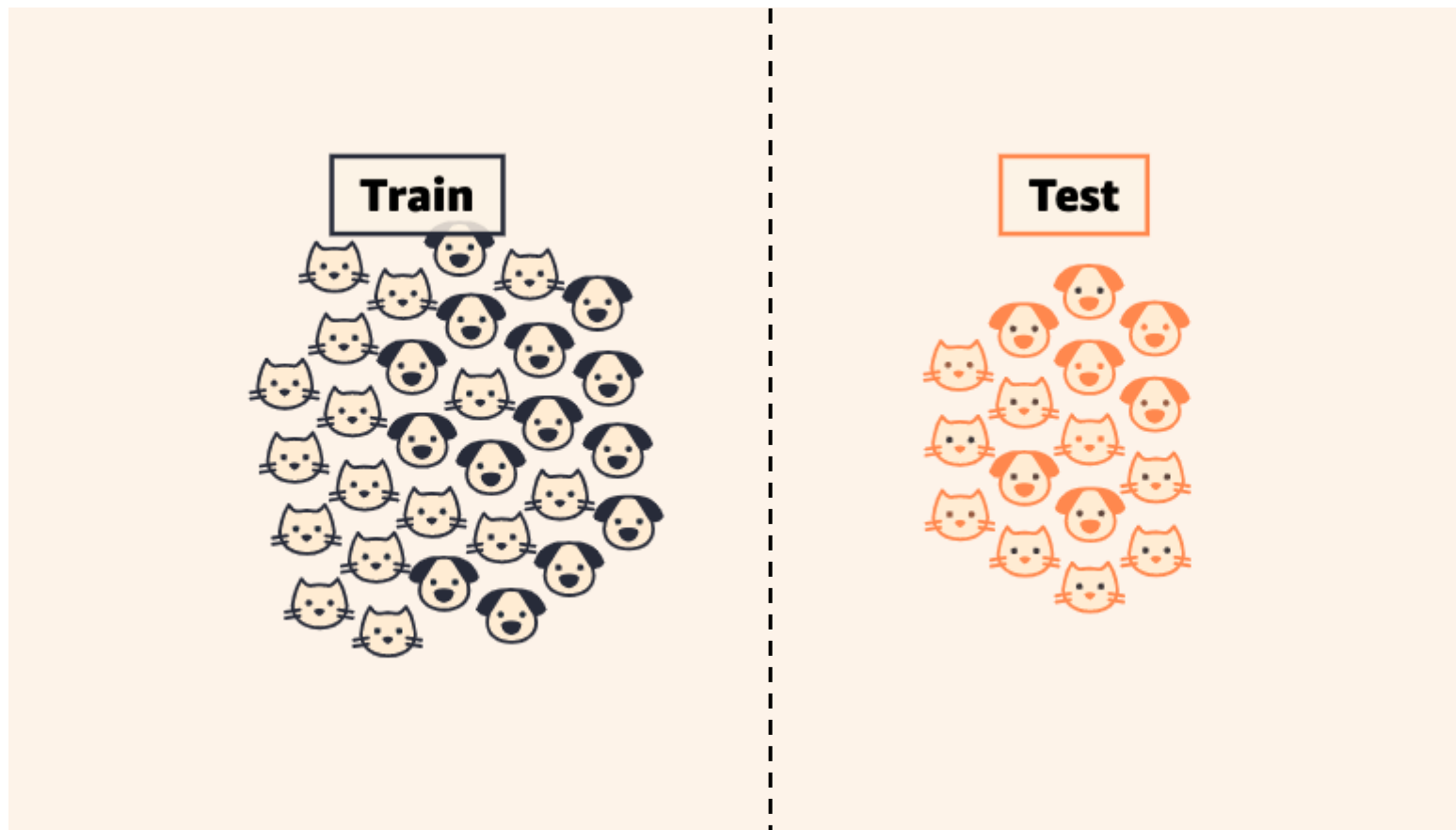
Test label (y_test)

```
(1437, 64)
```

```
(360, 64)
```

Data preprocessing with Scikit-learn

- Train set / Test set 나누기



KNN with Scikit-learn

- Train data로 KNN Classifier Model 학습시키기

```
from sklearn.neighbors import KNeighborsClassifier
```

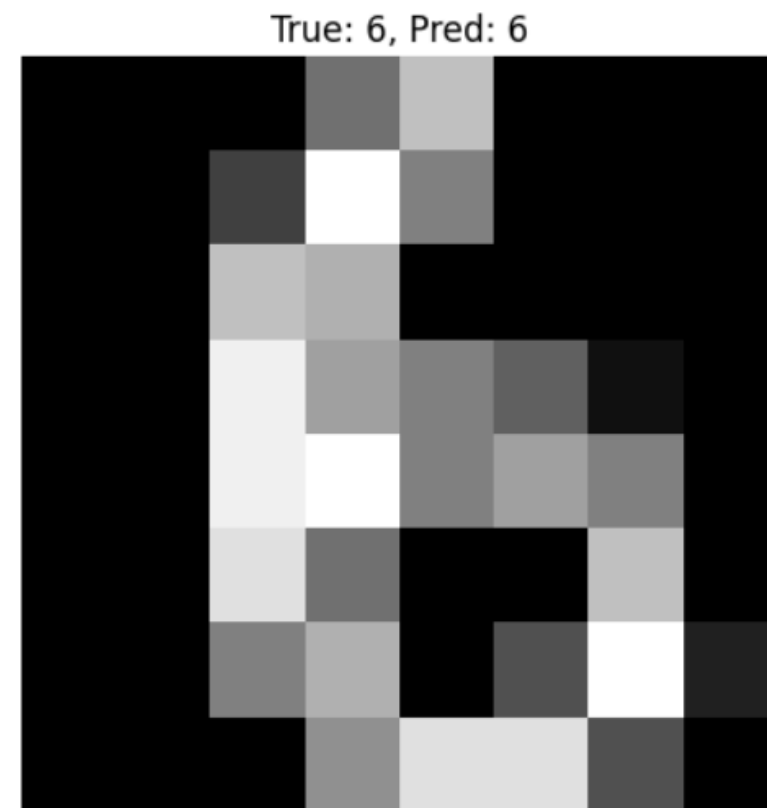
```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X_train, y_train)
```

Train data로 모델 학습시키기

```
y_pred = model.predict(X_test)
```

학습시킨 모델의 예측



True label: 6
Predicted label: 6

Evaluation with Scikit-learn

- Test data로 학습시킨 모델 성능 평가하기

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9833333333333333
```

Scikit-image (skimage)

- 이미지 처리에 특화된 python 이미지 라이브러리

```
from skimage import data, io, filters  
  
sfox = io.imread('fox.jpg')  
io.imshow(sfox)  
io.show()
```

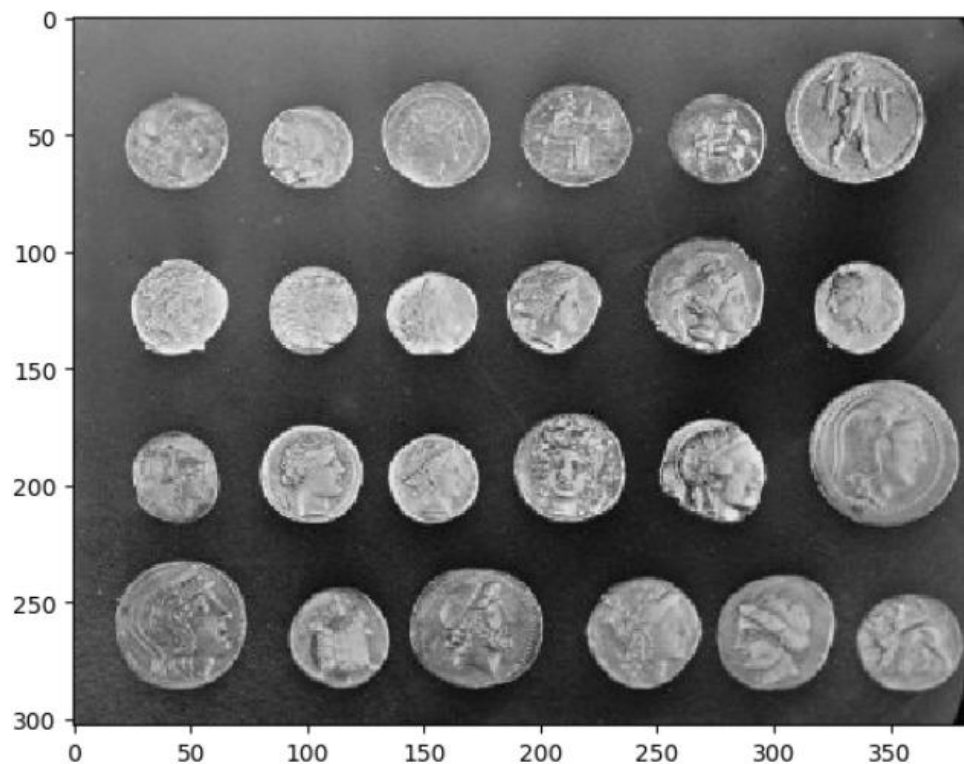


Scikit-image (skimage)

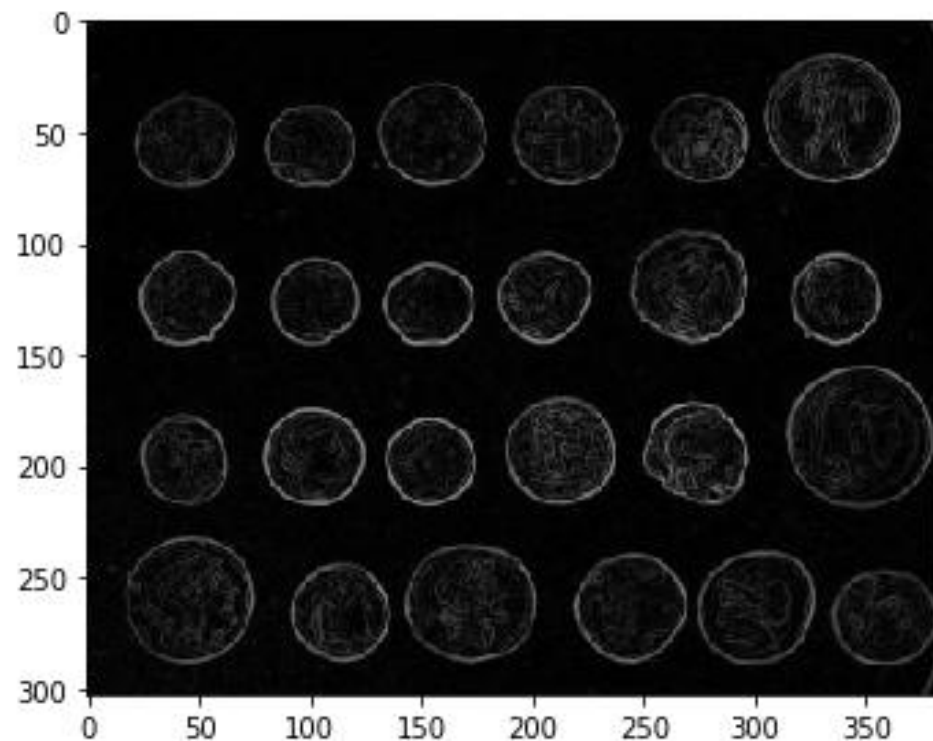
- 이미지 처리에 특화된 python 이미지 라이브러리

```
from skimage import data, io, filters
```

```
image = data.coins()  
io.imshow(image)  
io.show()
```



```
image = data.coins()  
# ... or any other NumPy array!  
edges = filters.sobel(image)  
io.imshow(edges)  
io.show()
```



Scikit-image (skimage)

- 이미지 처리에 특화된 python 이미지 라이브러리

```
from skimage import data
import matplotlib.pyplot as plt

img = data.astronaut()

plt.imshow(img)
plt.axis('off')
plt.show()
```



```
from skimage import data, color
from skimage.filters import threshold_otsu
import matplotlib.pyplot as plt

# 이미지 불러오기
image = data.astronaut()

# RGB → Grayscale
gray = color.rgb2gray(image)

# Otsu threshold 계산
threshold = threshold_otsu(gray)
    배경을 잘 분리할 수 있는 Threshold 선택
# Binary segmentation
binary = gray > threshold

# 결과 출력
plt.imshow(binary, cmap='gray')
plt.axis('off')
plt.show()

print("Otsu threshold:", threshold)
```



Otsu threshold: 0.388671875

Skimage Data Augmentation

- 학습 데이터를 늘리기 위해 기존 이미지에 다양한 변환을 적용해 새로운 이미지 생성
- skimage의 이미지 변환 기능 활용

